

# Pourush Kashyap

AI & Agentic AI Developer | Computer Science Student | Personal Portfolio Profile

---

## 1. About Pourush Kashyap

---

Pourush Kashyap is an AI and Agentic AI developer and an AI/ML engineer in the making. He is a Computer Science and Engineering student at CT University (batch of 2027) with a CGPA of 9.2, based in India. He is open to AI/ML opportunities, including internships and entry-level roles.

Pourush focuses on building intelligent software systems. His work spans machine learning, deep learning, computer vision, generative AI, retrieval-augmented generation (RAG), agentic AI and full-stack web development. He enjoys taking an AI idea all the way from a model or agent workflow to a working, deployable application with a proper frontend, backend and database.

His current direction is Agentic AI with LangGraph and MCP, RAG systems, multi-agent systems, AI evaluation and production reliability.

## 2. Contact Information and Links

---

Email: [pourushkashyap06@gmail.com](mailto:pourushkashyap06@gmail.com). Phone: +91 79863 55170. Location: India.

GitHub: <https://github.com/Pourushkashyap>. LinkedIn:

<https://www.linkedin.com/in/pourush-kashyap-68890a289/>. LeetCode:

[https://leetcode.com/u/\\_pourush2005/](https://leetcode.com/u/_pourush2005/).

Pourush is open to AI/ML opportunities and can be contacted by email, phone or LinkedIn.

## 3. Education

---

Pourush Kashyap is pursuing a B.Tech in Computer Science and Engineering at CT University, in the CSE batch of 2027, with a CGPA of 9.2.

He completed Class XII at Punjab Agricultural University with 92.4 percent, and Class X at Holy Child Foundation High School with 98 percent.

## 4. Technical Skills

---

**Programming languages:** Pourush works with Python, C++, JavaScript and TypeScript. He uses C++ for data structures and algorithms practice.

**Frontend development:** React, Tailwind CSS, Vite, Zustand, Redux Toolkit and Axios.

**Backend development:** FastAPI, Flask, Node.js and Express.js, including REST API design, authentication and third-party API integration.

**Databases:** MongoDB, PostgreSQL, pgvector and MySQL.

**Machine learning:** Scikit-learn, regression, classification, data preprocessing, feature engineering and model evaluation.

**Deep learning:** TensorFlow, neural networks, CNN, RNN, LSTM and Bidirectional LSTM models.

**Computer vision:** YOLOv8, object detection and image classification.

**Generative AI:** Large language models, Gemini, prompt engineering, embeddings and Hugging Face.

**RAG and AI agents:** FAISS, Chroma, vector search, LangChain, LangGraph, MCP (Model Context Protocol), tool calling, agent memory, planning and multi-agent systems.

**Deployment:** Docker, Docker Compose, CI/CD, cloud deployment and environment configuration.

**Engineering tools:** Git, GitHub, VS Code, Postman, debugging, testing, API design and system design.

## 5. Projects Overview

---

Pourush Kashyap has built eight main projects: Crime Scene Detection (computer vision), SilentSOS (audio machine learning and mobile), FinGrow (fintech with ML and generative AI), FitGenius AI (machine learning recommendations), CodePilot AI (agentic RAG for code), AgentForge (multi-agent software generation), Self-Healing Debugger (autonomous debugging) and his Portfolio Website. His most advanced work is in agentic AI: CodePilot AI, AgentForge and Self-Healing Debugger, all built with LangGraph and FastAPI.

## 6. Project: AgentForge

---

AgentForge is an autonomous AI website builder created by Pourush Kashyap. It turns a natural-language idea into a structured, tested web application. It falls under Agentic AI and multi-agent software generation.

### Problem it solves

A single LLM prompt can generate plausible code, but it does not reliably manage requirements, the contracts between frontend, backend and database, build failures or deployment. AgentForge turns app generation into a coordinated engineering pipeline with specialized planning and generation agents.

### How AgentForge works

The pipeline starts with the user's idea. A Requirement Analyzer converts the free-text idea into structured, testable requirements. A Master Planner coordinates the overall project plan. A Frontend Planner designs pages, components and implementation structure, a Backend Planner plans APIs, services and business logic, and a Database Planner designs models and persistence. Generator Agents then write the implementation code. Docker provides a reproducible build and test environment, the generated project is built and tested, failures can be fed back into the workflow, and a tested application is deployed.

### Technology stack

AgentForge uses LangGraph for orchestration, FastAPI for the backend, React, JavaScript and Tailwind CSS for the frontend, and Docker for build, test and deployment.

### Implemented features

Specialized planning and generation agents, structured requirement analysis, separate frontend, backend and database planning, a Docker-based build and test environment, and evaluation based on buildability, tests, deployment and structured agent artifacts.

### Future improvements

Planned improvements include feeding deployed-app failures back into planning, supporting extension of an existing generated application, pluggable planners for other technology stacks, better agent synchronization and parallel execution, and stronger dependency recovery and regression testing.

## 7. Project: Self-Healing Debugger

---

Self-Healing Debugger is an autonomous debugging and self-recovery system for Python projects, built by Pourush Kashyap. It belongs to Agentic AI, autonomous debugging and software reliability.

### Problem it solves

Traditional debugging requires manually reading a traceback, locating the source, diagnosing the root cause, writing a patch, running tests and checking for regressions. This system creates a controlled repair pipeline in which AI-generated patches are analyzed, validated, tested in a sandbox and risk-classified before they are applied.

### How Self-Healing Debugger works

The system runs the Python entry file and captures stdout, stderr and the exit code. When an error occurs, it parses the traceback into error type, message, file, line, function and stack trace. It uses the Python AST to locate the failing function and related called functions, and uses Gemini 2.5 Flash to diagnose the root cause. Related functions are retrieved through bounded AST traversal with cycle protection. The system then generates a replacement for the failing function while preserving its name and parameters, and validates syntax, signature preservation and configured dangerous operations. Candidate patches are applied only inside a temporary copied project, where project execution and pytest (when available) are run. Regression analysis decides whether the patch passes, retries or fails. The system classifies risk and pauses high-risk changes for human approval, and creates a .bak backup before writing an approved or low-risk fix.

### Technology stack

LangGraph for orchestration, FastAPI for the backend, React and JavaScript for the frontend, Gemini 2.5 Flash as the LLM, Python AST for code analysis, pytest for testing, and sandbox testing, regression analysis, risk classification and backup/rollback for safe recovery.

### Implemented features

Traceback parsing and source-context extraction, LLM-based root-cause analysis, AST-based related-code retrieval, patch generation with validation, sandbox testing and regression analysis, risk classification with human approval for sensitive changes, and a backup and rollback mechanism.

### Future improvements

Support for additional programming languages, better patch verification and mutation testing, stronger sandbox isolation and resource limits, persistent failure and repair memory, automatic rollback after failed health checks, and evaluation sets for patch correctness and safety.

## 8. Project: CodePilot AI

---

CodePilot AI is a multi-agent RAG coding assistant built by Pourush Kashyap for codebase analysis and natural-language code queries. It belongs to Agentic AI, RAG and code intelligence.

### Problem it solves

Large codebases are hard to understand by browsing manually. CodePilot AI gives natural-language access to files, functions, architecture and dependencies, while keeping answers grounded in the repository.

### How CodePilot AI works

A repository is ingested and parsed into retrievable units. The system builds dependency-aware context, splits code into chunks and generates Gemini embeddings. Guardrails and an intent router then choose the right path: specific questions use semantic retrieval, broad questions can use a project-level overview, and casual questions can skip retrieval entirely. An Answer Agent writes a grounded response, answer validation checks it before it is returned, and caching improves efficiency.

### Technology stack

FastAPI for the backend, LangGraph for orchestration, React, Vite and JavaScript for the frontend, Gemini embeddings for vectors, Supabase with HNSW indexing for vector search, PostgreSQL with connection pooling, JWT authentication, SHA-256 chunk identifiers and Docker.

### Implemented features

Multi-agent LangGraph orchestration, semantic code search using Gemini embeddings and Supabase with HNSW, repository-grounded natural-language code queries, project history, file browsing and code display, the `/ask`, `/ask/stream` (server-sent events) and `/ingest` endpoints, JWT authentication and Docker-based deployment packaging.

### Future improvements

More dependency-aware retrieval, improved cross-file reasoning, better evaluation datasets for code-grounded answers, incremental ingestion and cache optimization, and stronger security and sandboxing for code execution.

## 9. Project: FinGrow

---

FinGrow is a multi-service fintech platform built by Pourush Kashyap that combines peer-to-peer (P2P) lending, KYC, financial analysis and market insights. It spans fintech, full-stack development, machine learning and generative AI.

### Problem it solves

Financial workflows are often split across separate services for identity, lending, planning and market information. FinGrow brings these capabilities into one full-stack platform, with a separate Python financial-analysis service.

### How FinGrow works

Users register and log in through the web application, with JWT/cookies and bcrypt supporting authentication and protected access. KYC information can cover basic details, PAN, Aadhaar and bank verification. Lenders can create loan offers with amount and interest information, and borrowers can retrieve matching offers through backend APIs. Loan and transaction models and APIs provide the foundation for loan activity. Financial inputs are sent to a Flask service that performs savings analysis and Linear Regression-based estimation, and Gemini provides financial and investment insights. TradingView, Chatbase and MetaMask with ethers.js provide market charts, an assistant and wallet functionality.

## Technology stack

React, Vite, React Router and Context on the frontend; Node.js, Express.js and REST APIs on the backend; MongoDB with Mongoose for data; Python and Flask for financial analysis; Scikit-learn Linear Regression for estimation; Gemini for generative insights; JWT/cookies and bcrypt for authentication; and TradingView, Chatbase and ethers.js with MetaMask for integrations.

## Implemented features

The full-stack fintech platform structure, authentication and protected access, loan offer and matching foundations, a financial-analysis service with Linear Regression, and Gemini-based insights with external integrations.

## Future improvements

Completing the lender-borrower transaction lifecycle, a production repayment system with status updates, a dedicated credit-risk model with a proper dataset and evaluation pipeline, investor and lender dashboards, stronger KYC integrations and notifications, and better ML data collection, evaluation and monitoring.

# 10. Project: SilentSOS

---

SilentSOS is a voice-based intelligent safety system and a discreet mobile safety app disguised as a calculator, built by Pourush Kashyap. It combines mobile development, audio machine learning and full-stack engineering.

## Problem it solves

In a threatening situation, opening a conventional safety app or typing an emergency message may be difficult. SilentSOS listens for potential distress, classifies short audio samples and can trigger an emergency response discreetly.

## How SilentSOS works

When smart listening is activated, the app records a short audio clip about every five seconds. Audio is converted to 16 kHz mono WAV when required. Librosa extracts 40 MFCC features plus pitch, zero-crossing rate and RMS energy, producing a 43-dimensional representation. A TensorFlow stress model returns a danger-confidence value, and a configured confidence above 0.6 is treated as a potential danger event. Emergency processing then obtains GPS coordinates and sends SOS information to trusted contacts with a map location and the audio attachment. Node.js and Express handle emergency processing, MongoDB stores alert information and Socket.IO supports real-time location updates.

## Technology stack

React Native and Expo with JavaScript for the mobile app; FFmpeg and Librosa for audio processing; TensorFlow with a Bidirectional LSTM for deep learning; Python and Flask for the ML API; Node.js and Express.js for the emergency backend; MongoDB for storage; and Socket.IO for real-time updates.

## Implemented features

A discreet React Native mobile workflow, audio preprocessing and feature extraction with FFmpeg and Librosa, a Bidirectional LSTM inference pipeline, a Node.js/Express emergency workflow with MongoDB alert logging, and SOS dispatch with GPS location and audio attachment.

### Future improvements

Running validated inference closer to the device to reduce latency and improve privacy, combining vocal stress with keyword detection, motion and contextual signals, better background execution on Android and iOS, secure production APIs with HTTPS, production SMS and emergency-service integrations, and a trusted-contact and emergency dashboard.

## 11. Project: Crime Scene Detection

---

Crime Scene Detection is an image-based crime-scene analysis web application built by Pourush Kashyap as his internship project at Solitaire Infosys. It belongs to computer vision, deep learning and full-stack development.

### Problem it solves

Crime-scene images can contain visual evidence such as bloodstains, knives and firearms. The application gives a web workflow that accepts an image, performs object detection and returns predictions dynamically.

### How it works

The user uploads a crime-scene image through the React interface. The frontend sends the image to a Flask backend through a REST API, which passes it to the computer-vision inference workflow. YOLOv8 processes the image for object detection, with supported targets including bloodstains, knives and firearms. The predictions are returned and displayed dynamically in the React result interface.

### Technology stack

React.js and Tailwind CSS for the frontend, Python, Flask and REST APIs for the backend, and YOLOv8 object detection for computer vision.

### Implemented features

An image upload and dynamic prediction workflow, a React frontend integrated with a Flask REST API, and YOLOv8-based object detection for the supported visual targets.

### Future improvements

Expanding the training data and supported classes, adding formal precision, recall and mAP reporting, improving robustness to lighting, blur, camera angle and image quality, better confidence-threshold configuration and bounding-box visualization, model versioning with reproducible deployment, and secure image handling with audit logging if persistent storage is added.

## 12. Project: FitGenius AI

---

FitGenius AI is a personalized diet and workout recommendation system built by Pourush Kashyap using user-specific inputs and a machine-learning workflow.

### Problem it solves

Generic fitness plans may not account for an individual's body metrics, goals and preferences. FitGenius AI uses these inputs to produce a tailored recommendation.

### How it works

The system collects body metrics, goals and preferences through a React interface, preprocesses the input, performs feature engineering and runs a Scikit-learn model. The prediction is turned into a personalized diet and workout recommendation and returned to the React interface through a Flask API.

### Technology stack

React and JavaScript for the frontend, Flask for the ML API, and Scikit-learn for the model, with a preprocessing, feature engineering and prediction pipeline.

### Future improvements

Expanding and validating the training dataset, adding stronger validation and model metrics, improving personalization and constraints, adding recommendation history and feedback, and monitoring recommendation quality and drift.

## 13. Project: Portfolio Website

---

The portfolio website is Pourush Kashyap's personal site, built with React 18, Vite and Tailwind CSS. It presents his professional identity, projects, skills, journey, background and contact information, and includes custom canvas-based 3D visual scenes and an AI assistant interface.

The site has a Home page with an introduction, featured projects and a system snapshot; an About page on his engineering philosophy and current direction; a Projects page with case studies and architecture; a Skills page; a Journey page showing his progression from full-stack development toward autonomous AI systems; a Background page with education, experience, achievements and current focus; and a Contact page with email, phone, social links, resume and the assistant entry point. Each project case study covers overview, problem, workflow, implementation and roadmap. The site uses reusable React components and structured data modules, and the assistant interface is being connected to an intelligent RAG backend so visitors can ask questions about Pourush.

## 14. Experience and Leadership

---

Pourush completed an internship at Solitaire Infosys, where he built the Crime Scene Detection project using React, Flask and YOLOv8.

He is a Coordinator of Udaan at CT University.

## 15. Achievements

---

Pourush Kashyap won the Best Startup Award at Code Crafter 2.0. He has twice been the winner of the LeetCode Coding Competition at CT University, and earned a LeetCode T-shirt achievement. According to his resume, he has solved more than 1,200 Data Structures and Algorithms problems across LeetCode, GeeksforGeeks and Coding Ninjas, using C++. His problem-solving areas include arrays, strings, trees, graphs, dynamic programming, binary search and greedy algorithms.

## 16. Learning Journey

---

In 2023, Pourush focused on JavaScript, React, Node.js and MongoDB. In 2024, he moved to Python, machine learning, Pandas and Scikit-learn. In 2025, he learned TensorFlow, deep learning, LLMs and RAG. In 2026, he is working on LangGraph, MCP, agents, multi-agent systems and Docker. This path took him from full-stack web development to building autonomous AI systems.

## 17. Career Goals and Roles of Interest

---

Pourush is looking for roles in three areas. An AI/ML Intern role would involve ML models, data preprocessing, model evaluation and Python. An AI Engineering Intern role would involve LLM applications, RAG, agents and LangGraph. A Full-Stack AI role would involve React, FastAPI or Node, databases and AI integration.

His current learning focus is AI/ML, generative AI, agentic AI, LangGraph, MCP, RAG, AI-powered software systems, and AI evaluation and production reliability.